

# LLM委员会考试系统

基于多Agent的自动考试

## 评估Agent最终裁定

Evaluation ID: b126f7e1-44f0-433e-850b-66f8765477ee

5.0/10

考官问题平均分

主席  
评估

理解核心概念但存在关键应用局限，处于  
表面理解与深入理解之间

基于两个深度测试问题的综合评分（平均5.0/10），学生  
法实现（大整数乘法的手动模拟）上展现出基础理解能力  
实现主要逻辑，但在关键细节的精确解释上存在明显不足  
（5.4分）显示学生对竖式乘法中数位索引（ $p1, p2$ ）的解  
解模糊，未能清晰建立算法实现与数学原理的精确映射；  
分）表明学生对防御性编程模式（`lstrip('0')` or `'0'`）的达  
存在关键性混淆，无法准确识别触发`'or'`分支的具体场景  
然而不知其所以然'的状态，反映出学生具备代码实现的  
但缺乏对底层机制的深入把握和严谨的边界条件分析能力

### 识别出的知识漏洞:

竖式乘法中数位位置的精确对应关系（ $p1, p2$ 与个位、十位等的

字符串操作边界条件的精确分析（特别是`lstrip('0')`产生空字符

防御性编程中'短路求值'逻辑与异常处理的因果关系

算法实现与数学原理的形式化对应能力

### 学习建议:

- 通过数学推导重新梳理竖式乘法：手动模拟两个多位数相乘，  
止数之和的十位和个位在最终数组中的确切位置。建立`'i'`和

各维度详细评分（三阶段委员会流程）

每个问题经过多教师独立评分 → 同行评议校准 → 主席综合裁定

1

在你的代码中，变量`p1`和`p2`分别被赋值为`i + j`和`i + j + 1`。请解释这两个变量在...

5.4

C  
(及格)

置信度: 低 教师数: 4 一致性: 中

2

代码的最后使用了表达式`res\_str.lstrip('0') or "0"`来返回最终结果。请解释：当输入的`n`...

4.5

D  
(不及格)

置信度: 低 教师数: 4 一致性: 中

1 教师委员会独立评分

kimi-k2.5

6.0/10

kimi-k2-0905-preview

3.0/10

kimi-k2-turbo-preview

3.0/10

kimi-k2-thinking

4.5/10

最严格: kimi-k2-0905-preview | 最宽松: kimi-k2.5

3 委员会主席详细评语

**\*\*最终得分\*\***: 4.5/10

**\*\*置信度\*\***: 中 (评分差异度方差3.0, 存在中分歧)

**\*\*评定等级\*\***: D(<5)

**\*\*委员会主席评语\*\***:

- **\*\*综合评价\*\***: 该答案对代码的防御性意图有初步感知，但在核心概念上存在关键性混淆，未能准确回答“空字符串产生的具体条件”这一技术要点，且对`result`数组构造过程的描述流于表面，缺乏精确的因果逻辑链条。

- **\*\*主要优点\*\***:
  1. 正确识别`or "0"`的防御性编程意图，表明对代码健壮性有一定认识；
  2. 能够区分输入为"0"与不为"0"时的不同表现，方向性判断基本正确；
  3. 提及`lstrip('0')`会移除前导零的操作特性。
  
- **\*\*关键不足\*\***:
  1. **\*\*概念混淆\*\***: 错误地将`result[0]`为0（无前向进位）与“产生空字符串”关联。实际上，仅有前导零被移除不会导致空字符串，只有当`res\_str`全部为'0'（即计算结果为0）时，`lstrip('0')`才会返回空字符串`""`；
  2. **\*\*答非所问\*\***: 未明确回答题目设定的核心问题——在输入不为"0"的前提下，何种情况会导致空字符串（理论上不应发生），反而引入“计算异常”等模糊假设；
  3. **\*\*论证缺失\*\***: 未结合`result`数组的具体构造过程（初始化、双重循环累加、进位传播）论证为何非零输入必然产生非零结果，缺乏技术细节支撑。
  
- **\*\*得分依据\*\***:

本裁定倾向于**\*\*教师D (kimi-k2-thinking)\*\***的评判立场。虽然学生表现出对边界情况的敏感度，但混淆“前导零移除”与“全零字符串”是致命的概念错误，且未紧扣`result`数组的构造机制进行推导。教师A（6.0分）过于宽容，未能充分惩戒核心概念错误；教师B与C（3.0分）虽严格但忽略了学生对防御性编程的正确识别。4.5分体现了对“部分理解”的认可，同时强调关键知识缺陷的严重性。
  
- \*\*学习建议\*\***:
  1. **\*\*精确理解字符串操作\*\***: 通过实验验证`"00123".lstrip('0')`与`"000".lstrip('0')`的区别，明确空字符串产生的唯一条件是“字符串全部由被剥离字符组成”；
  2. **\*\*构建完整因果链\*\***: 针对此类问题，按“输入条件→数组构造→数值计算→字符串转换→边界处理”的链条逐步推导，避免从中间状态（如`result[0]=0`）直接跳跃到结论；
  3. **\*\*区分理论与防御\*\***: 在分析代码时，应明确区分“理论上的不可能事件”（非零输入产生零结果）与“防御性编程的实际目的”（处理合法但特殊的输入如"0"），避免将两者混为一谈；
  4. **\*\*深入算法细节\*\***: 重新梳理大整数乘法的标准实现，重点理解`result`数组的初始化长度、位乘积累加机制以及进位处理如何确保非零输入必然产生非零最高位。

学生回答预览

当result数组的构造过程中，如果最高位没有产生进位，result[0]保持为0，那么res\_str会以'0'开头。调用lstrip('0')会移除所有前导零，如果结果恰好是"0"（比如两个输入都是"0"，但题目排除了这种情况），或者理论上如果计算异常导致全零.....实际上在本题中，如果输入不为"0"，result数组至少有一个非零位，lstrip后不会为空，但代码中使用or "0"是

评分分布

|           |   |
|-----------|---|
| 优秀 (9-10) | 0 |
| 良好 (7-8)  | 0 |
| 中等 (5-6)  | 1 |
| 待改进 (<5)  | 1 |

评估流程说明

|             |              |              |
|-------------|--------------|--------------|
| 1           | 2            | 3            |
| 多教师<br>独立评分 | 同行评议<br>交叉校准 | 主席综合<br>最终裁定 |

开始新的评估